

# P\_Web 295 - Passion Lecture ( Backend )

---

Auteurs : Jonathan Junod et Néo Darbellay Chef de projet : Grégory Charmier (GCR) Dates : 28.04.2026 - 26.05.2026  
Module : C295

## Introduction

Dans ce rapport, nous parlerons de la manière par laquelle l'équipe a organisé et divisé les tâches ainsi que l'outil que nous avons utilisé afin de se faire. Nous aborderons une partie analyse de l'application et nous parlerons de l'architecture de la base de données. Nous présenterons un tableau des routes mises en place ainsi que leurs fonction. Enfin nous détaillerons l'organisation du code et de l'équipe par rapport à la réalisation des tâches. Nous concluons évidemment par une conclusion générale ainsi qu'un cours paragraphe par personne sur une brève conclusion personnelle.

Ce projet a pour objectif de concevoir et développer une application web complète permettant la gestion de livres en leur attachants des commentaires, des catégories, des éditeurs ainsi qu'au moins un écrivain. Cette application repose sur une architecture MVC séparant le frontend et le backend. Dans cette partie du projet, nous allons donc développer une API REST, une base de donnée de type SQL ainsi qu'un système d'authentification des utilisateurs.

Ce morceau de projet consiste donc en améliorer et compléter un frontend réalisé précédemment en Vue.js au cours du module 294, puis à développer une API REST complète avec AdonisJS, MySQL ainsi qu'une multitudes d'autres outils.

L'application développée permettra aux utilisateurs de consulter les ouvrages disponibles ainsi que leurs catégories. Les utilisateurs connectés peuvent ajouter, modifier ou supprimer leurs propres livres. Ils peuvent aussi ajouter des commentaires accompagnés d'une note de 0 à 5 sur les livres.

## Analyse

### Planification des tâches

Afin de travailler efficacement, nous avons décidé d'utiliser un outil de gestion de projet très répandu dans le monde de la programmation. En effet, afin de rester organisé, nous avons utilisé GitHub Project. Cet outil nous permet de notamment

- Créer des tâches.
- Définir qui est responsable de quelle tâche.
- Déplacer des tâches dans des catégories en fonction de leur état.
- Et bien d'autres choses dont nous n'avons pas eu besoin jusqu'à présent.

En effet, à l'aide de GitHub Project, nous avons pu créer des tâches concernant le projet. Nous avons essayé de maintenir une granularité d'environ 45 minutes à 1 heure pour ces tâches, c'est à dire que n'importe laquelle de ces tâches devrait pouvoir être réalisée en ces délais. Nous aurions pu décider de créer des tâches plus précises afin d'être ensuite plus efficace (une personne par demi-tâche afin d'expédier ce qui est bloquant tout de suite), mais nous avons décidé qu'une période par tâche serait un bon compromis. Ceci car le temps à disposition afin de réaliser le projet est très limité et nous ne souhaitons pas perdre trop de temps là dessus.

Nous avons aussi parlé de l'assignation des tâches par GitHub. Cet outil merveilleux, nous permet en effet d'assigner et de s'assigner à plusieurs tâches. Les concernés sont alors notifié par mail (sauf si cela à été explicitement désactivé par le/la concerné(e)) et peuvent consulter la tâche à réaliser.

Enfin GitHub Project nous permet aussi de déplacer des tâches en fonction de leur avancée dans diverses colonnes. GitHub nous mets directement plusieurs colonnes à disposition que voici.

1. **Backlog** : cette colonne indique que la tâche figure dans le backlog et devra être réalisée. Pour le moment, les circonstances ne permettent pas de réaliser la tâche. Par exemple, nous ne pouvons pas commencer à réaliser les *modèles* et *migrations* Adonis tant que les documents MCD et MLD n'ont pas été fournis.
2. **Ready** : Dans cette catégorie, les tâches sont prêtes à être prises afin d'être réalisées, tout ce qu'il manque est un (ou plusieurs) participant assigné (et optionnellement motivé).
3. **In progress** : ces issues sont en train d'être réalisé, tout n'est plus qu'une question de temps à ce moment. Un ou plusieurs développeur travaille activement dessus.
4. **In review** : Ici, les activités sont en train d'être testées puis validées, une tâche laissée de côté peut se retrouver ici.
5. **Done** : Enfin, ma colonne préférée, Done signifie que la présente tâche est terminée et validée, il n'y a donc plus aucun travail à fournir pour ce point.

## Analyse de l'API REST

### Catégories

| Verbe HTTP | URI                 | JSON             | Auth |
|------------|---------------------|------------------|------|
| GET        | /api/categories     | -                | Non  |
| GET        | /api/categories/:id | -                | Non  |
| POST       | /api/categories     | {"label":"Dogs"} | Oui  |
| PUT        | /api/categories/:id | {"label":"Cats"} | Oui  |
| DELETE     | /api/categories/:id | -                | Oui  |

### Éditeurs

| Verbe HTTP | URI              | JSON           | Auth |
|------------|------------------|----------------|------|
| GET        | /api/editors     | -              | Non  |
| GET        | /api/editors/:id | -              | Non  |
| POST       | /api/editors     | {"name":"Noé"} | Oui  |
| PUT        | /api/editors/:id | {"name":"Néo"} | Oui  |
| DELETE     | /api/editors/:id | -              | Oui  |

### Livres

| Verbe HTTP | URI            | JSON                                                                                                                                                                                                                                                        | Auth |
|------------|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| GET        | /api/books     | -                                                                                                                                                                                                                                                           | Non  |
| GET        | /api/books/:id | -                                                                                                                                                                                                                                                           | Non  |
| POST       | /api/books     | { "title": "Something strange happened", "numberOfPages": 319, "pdfLink": "/false", "abstract": "Just an easter egg for Noe to find", "editionYear": "2025", "imagePath": "/true", "userId": 1, "writerId": 2, "editorIds": [3,4], "categoriesIds": [1,2] } | Oui  |
| PUT        | /api/books/:id | { "title": "Something strange happened", "numberOfPages": 319, "pdfLink": "/false", "abstract": "Just an easter egg for Noe to find", "editionYear": "2025", "imagePath": "/true", "userId": 1, "writerId": 2, "editorIds": [3,4], "categoriesIds": [1,2] } | Oui  |
| DELETE     | /api/books/:id | -                                                                                                                                                                                                                                                           | Oui  |

## Users

| Verbe HTTP | URI                 | JSON | Auth |
|------------|---------------------|------|------|
| POST       | /api/users/register | -    | Non  |
| POST       | /api/users/login    | -    | Non  |
| POST       | /api/users/logout   | -    | Oui  |
| GET        | /api/users          | -    | Oui  |
| GET        | /api/users/:id      | -    | Oui  |
| PUT        | /api/users/:id      | -    | Oui  |
| DELETE     | /api/users/:id      | -    | Oui  |
| GET        | /api/me             | -    | Oui  |

## Docs

| Verbe HTTP | URI      | JSON | Auth |
|------------|----------|------|------|
| GET        | /swagger | -    | Non  |
| GET        | /docs    | -    | Non  |

PAGES : 1

## Analyse de la DB

À présent, nous allons parler de la base de données et plus précisément de sa conception.

Concernant la base de données, nous avons essayé de faire au plus simple, sans sacrifier les performances de l'application.

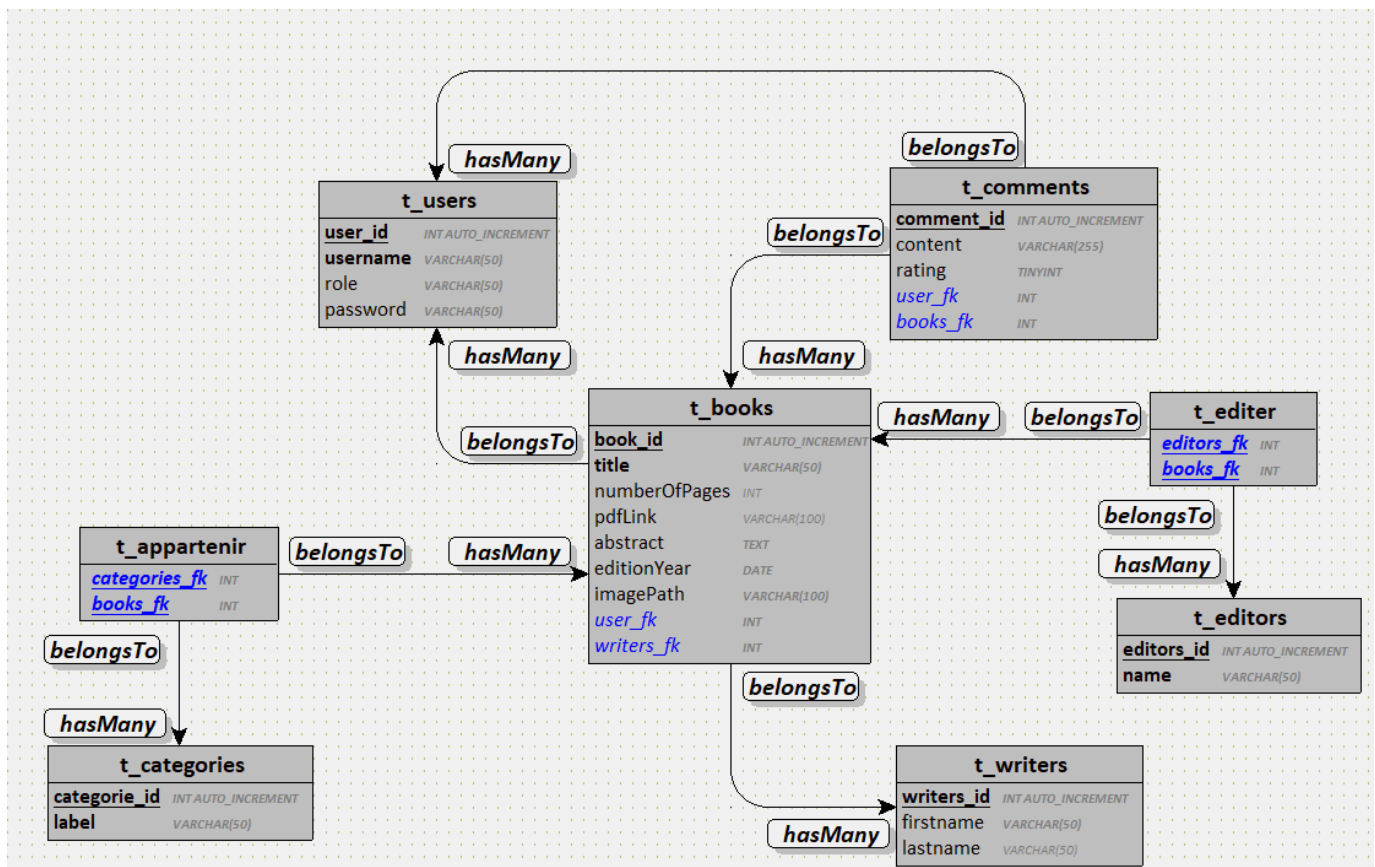
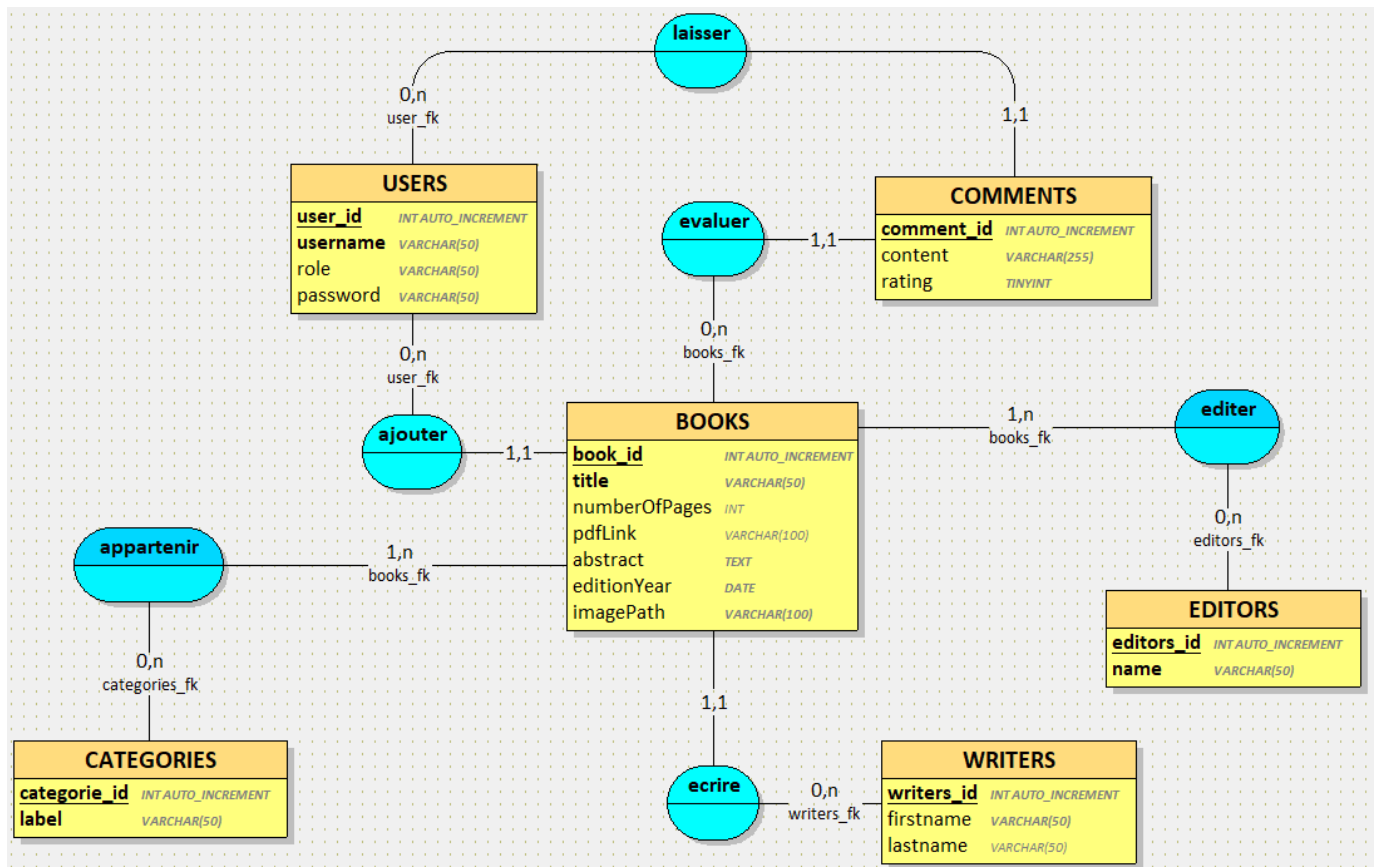
La table `t_book` se coeur du schéma est . C'est là que l'on stocke tout le contenu des livre, c'est à dire

- le titre
- le résumé
- le nombre de pages
- l'année
- les liens pour la couverture et le PDF.

Afin de savoir qui a ajouté le livre sur le site, nous avons mis une simple relation belongsTo qui relie le livre à l'utilisateur. Nous avons aussi ajouté une table `t_writer` avec le même type de relation afin de lier un livre avec un écrivain.

Plus compliqué ensuite, les relations entre books et leurs éditeurs ainsi que leurs catégories. Puisqu'un livre peut avoir plusieurs éditeurs et catégories, nous avons mis en place 2 tables (`t_appartenir` et `t_editer`).

Ainsi lorsqu'un livre est créé, nous devons créer autant d'enregistrement dans les tables `belong` et `edit` qu'il y a d'éditeurs et de catégories attachées à ce livre.



## Relations des modèles

Nous allons détailler un peu plus ces relations entre les tables de la base de données. En effet, il faut tout d'abord savoir qu'Adonis propose quelques relations prédéfinies telles que les 2 que nous avons utilisées. Nous parlons ici des relations **belongsTo** et **hasMany**, ces dernières sont complémentaires et conviennent parfaitement à notre architecture db.

Concernant la partie *simple* des relations, nous avons mis en place la relation **HasMany** -> **BelongsTo** depuis Books vers Comments, Écrivains vers Books, Users vers Books et enfin Users vers Comments. Par exemple, un utilisateur peut créer plusieurs commentaires.

Maintenant vient la partie plus complexe qui nous a posé problème durant la mise en place du CRUD des livres. Les relations entre Books - Catégories et Books - Éditeurs sont de type 1, n - 0, n. En effet, un livre doit appartenir à au moins une catégorie ainsi qu'un éditeur. Cependant ce même livre peut appartenir à plusieurs catégories et éditeurs. Et étant donné qu'une catégorie peut posséder entre 0 et n livre de même qu'un éditeur peut éditer entre 0 et n livres, nous aurions eu un problème de foreign key si nous nous étions contenté de 2 tables (et MySQL n'aurait de toutes manières pas accepté cet affront).

C'est pourquoi nous avons mis en place un table intermédiaire qui possèdera les 2 fks. La relation se fait donc ainsi : Books Hasmany enregistrements dans la table intermédiaire et cette même table Belongsto un seul livre et en même temps un seul éditeur/catégorie. Évidemment un éditeur/catégorie Hasmany enregistrements dans la table intermédiaire.

## Schéma d'interaction frontend/backend

Parlons maintenant des relations à l'intérieur de l'application. Nous pouvons laisser la théorie concernant l'architecture de la base de données de côté pour le moment. Concernant le procédé, voici comment cela se déroule.

1. L'utilisateur demande une page (par exemple la page des livres).
2. La partie frontend comprend le clic de l'utilisateur et demande les infos de ladite page au backend.
3. Pendant que le backend réfléchit, la partie frontend de l'application dessine déjà la page sans aucun livre (donc juste nav, background, layout général).
4. La partie backend reçoit la requête GET et va chercher tous les livres classés par catégories dans la fameuse base de données à l'aides des modèles (fournis par LucidORM).
5. La base de données donne la liste de livre ordonnée et classée au serveur (qui est le backend).
6. Le backend revoie les infos demandées accompagnées d'un code de status 200 (ou une erreur avec le code approprié selon ce qu'il s'est passé).
7. Le frontend reçoit les infos et assigne une variable de type **ref()** à ces dernières.
8. La variable **ref()** se trouve être en fait, un type spécial de variable qui est interactif. La page va donc ajouter les données de la variable **ref()** en temps réel.

Et l'utilisateur voit donc sa page entière.

## Réalisation

### Authentification et rôles

Pour l'authentification, nous avons séparé les utilisateurs en deux groupes :

- Les Users
- Les Admins

Les **Users** peuvent uniquement éditer et supprimer leur propre livres, et créer des livres. Les **Admins** peuvent faire du CRUD sur toutes les tables (Editeurs, Catégories, Livres, Utilisateurs)

L'authentification en elle même se fait en faisant un POST à **/api/users/login** dans le backend et en allant à la page **/login** sur le front end Vu que nous n'avions pas le temps pour faire mieux, nous avons mis le token dans le localStorage.

Les routes C, U et D du CRUD pour tout (sauf `/register` et `/login`) sont protégés par `router.use(middleware)`

## Mesures de sécurité

Comme nous en avons parlé au chapitre précédent, la sécurisation a été mise en place de tel : Les routes C U et D du CRUD ont été sécurisées, sauf `/api/users/register` et `/api/users/login`, en utilisant le middleware d'authentification. Seul les Admins peuvent avoir accès au CRUD complet de chaque table. Par contre, les Users peuvent créer des livres, et modifier/supprimer les leurs.

## Test

Nous avons testé le **Backend** en utilisant Bruno. La collection Bruno est disponible dans le repo Github sous `/Bruno`.

Bruno est un outil très pratique dans le milieu du backend, en effet, ce dernier nous permet de tester les routes et requêtes sans avoir à monter le frontend. De plus, ce dernier est muni d'un outil encore plus pratique, laissez-moi introduire *le runner*. Le Runner (petit bonhomme en haut à droite), lance toutes les requêtes désirées, nous pouvons programmer lesquelles exécuter et dans quel ordre, ce qui nous permet d'éviter le login -> logout et les autres ne fonctionnent plus. Ce que nous vous conseillons est donc de lancer *login* à la main au départ et de stocker le token dans la variable correspondante (à trouver directement dans les variables de la collection). Ensuite, vous pourrez lancer le runner SANS les requêtes login et logout, voyez si tout fonctionne. Enfin testez logout seul.

Une autre manière simple de déboguer les routes avec Adonis est de lancer la commande `node ace list:routes` dans le **Backend**. Cela retourne une liste de tous les verbes HTTP, toutes les routes, les Handlers (CONTROLLER.FUNC) et le middleware. Nous avons testé le **Frontend** en vérifiant que chaque page fonctionne comme il le faut.

## Conclusion

### Organisation du groupe de la gestion du code

Dans ce chapitre, nous revenons sur [le paragraphe](#), et comme cité précédemment, nous avons utilisé GitHub Project afin de nous organiser. En effet, le modèle de gestion de projet de type *kanban*, nous a permis de travailler efficacement sans se casser la tête. Ce modèle privilégie en effet la représentation visuelle et la facilité à se retrouver dans l'avancée du projet. Comme décrit plus haut, le modèle est constitué de 5 colonnes/catégories permettant de classer les tâches selon leur état.

Nous avons donc mis en place une façon de s'organiser plutôt stricte sans avoir à se le dire explicitement. Dans le principe, à chaque fois qu'une personne termine ou prend une tâche, cela est dit par oral. De plus, lorsqu'une personne assigne une tâche à un autre membre du groupe, le membre assigné se retrouve notifié d'un email dans sa boîte de réception. Cela fonctionne aussi si une tâche assignée à une personne est close par une autre, le propriétaire de la tâche reçoit un mail.

### Conclusion Générale

Ce projet paraissait simple, mais c'est avoué compliqué vers la fin, car nous n'avions presque plus de temps. Nous avons réussi à tout finir, malgré notre retard. Notre rythme de travail était correct, mais nous avons eu des soucis concernant un membre de l'équipe qui était malade pendant une partie conséquente du projet.

### Conclusion Personnelle

## Jonathan

Ce fut un demi-projet plutôt simple sur le papier, plutôt compliqué sur sa fin. Selon moi, avoir 2 après midis de plus aurait été top (et nous n'aurions pas eu à terminer le rapport à 21h30). Le fait que 2 projets s'assemblent était très intéressant, surtout le fait de travailler en binômes.

Avoir commencé par le frontend fut selon moi, le choix le moins confus pour les élèves. La seule remarque que je trouve serait de plus insister sur les routes durant le module frontend (1ere partie du projet). L'analyse des routes en verbes http est très importante pour la suite et un seul souci peu poser de graves pertes de temps par la suite.

Autre chose pas forcément liée, mais que je trouve quand même intéressante à dire : *Le PBulle Adonis est affreux et je n'ai, personnellement, rien vraiment appris durant ce temps à part à quel point un projet peut être douloureux. La critique est méchante et je suis conscient que les enseignants se donnent du mal pour faire au mieux, c'est pourquoi je propose qu'au lieu de ce P\_Bulle sans théorie (et c'est le point principal qui fait de ce projet une chose compliquée pour les apprentis), et puisque (j'imagine) il est nécessaire que ce soit un projet, l'année prochaine, les élèves commencent par 294, puis 295, puis bulle Adonis. Le but serait de montrer qu'Adonis est puissant et peut gérer les vues. Sinon, pourquoi ne pas introduire un module \_Express, puisque nous avons eu à faire à des application de ce genre sans n'avoir jamais su comment cela fonctionne. Dans le cas où le projet n'est pas nécessaire, j'ai pensé à rallonger le projet 294 - 295, afin que les participants aient plus de temps pour les détails (CRUD commentaire et autres).*

## Néo

J'ai trouvé ce projet fort intéressant, mais ai trouvé très dommage que je n'ai pas pu profiter de tout le temps à ma disposition à cause de mes absences maladies. Si je devais refaire ce projet à l'avenir, j'espère que je ne serai pas aussi malade, car cela a causé de gros soucis de temps.

## Planification du projet

La [planification](#) que nous avons fait pour ce projet était bien, mais aurait pu être séparée en plusieurs partie encore plus petites. Vu que nous avons fait la planification sur GitHub Project, vous pouvez consulter l'intégralité des tâches à [ce lien](#)